ABSTRACT

Title of Thesis: A FRAMEWORK FOR BENCHMARKING GRAPH HARDWARE ACCELERATORS

Kent Daniel O'Sullivan
Master of Science, 2024

Thesis Directed by: Professor William Regli
Department of Computer Science

ABSTRACT HERE

A FRAMEWORK FOR BENCHMARKING GRAPH HARDWARE ACCELERATORS

by

Kent Daniel O'Sullivan

Thesis submitted to the Faculty of the Graduate School of the
University of Maryland, College Park in partial fulfillment
of the requirements for the degree of
Master of Science
2024

Advisory Committee:
William Regli, Chair/Advisor
Mohammad Hajiaghayi
Brian Pierce
Laxman Dhulipala

© Copyright by
Kent O'Sullivan
2024

Preface

If needed, PREFACE HERE.

Foreword

If needed, FOREWORD HERE.

Dedication

If needed, DEDICATION HERE.

Acknowledgments

ACKS HERE

Table of Contents

Preface ii

Foreword ii

Dedication ii

Acknowledgements ii

Table of Contents iii

List of Tables v

List of Figures vi

List of Abbreviations vii

Chapter 1: Introduction 1
- 1.1 Intelligence and Security Context . . . 2
  - 1.1.1 Use Cases for Intelligence and Security Problems . . . 2
  - 1.1.2 Intelligence and Security Datasets . . . 17
- 1.2 Graph Data Structures . . . 18
  - 1.2.1 Notation . . . 18
  - 1.2.2 Graph Representation . . . 19
  - 1.2.3 Graph Primitives . . . 26
  - 1.2.4 General Characteristics . . . 29
  - 1.2.5 The Intelligence and Security Graph Problems . . . 33
  - 1.2.6 Processing Bottlenecks . . . 39
  - 1.2.7 Summary . . . 55
- 1.3 Graph Hardware Acceleration . . . 55
- 1.4 Benchmarking Graphs . . . 57
- 1.5 Thesis Outline . . . 57

Chapter 2: Related Work 58

Chapter 3: Graph Artificial Intelligence Benchmarking 59
- 3.1 Problem Statement . . . 59
- 3.2 Methodology . . . 62

3.2.1 Benchmark Design Principles . . . 62
3.2.2 Benchmark Components . . . 63
3.2.3 Metrics . . . 64
3.2.4 Benchmarking Graph Applications . . . 67
3.2.5 Benchmarking Artificial Intelligence Applications . . . 67
3.2.6 Empirical Methods in Computer Science . . . 67
3.2.7 Principles for designing a Graph-Based AI Benchmark . . . 67
3.3 Experiment Design . . . 67
3.3.1 Collecting Metrics . . . 68
3.4 Results . . . 68
3.5 Discussion . . . 68

Chapter 4: Benchmarking Community Detection **68**
4.1 Problem Statement . . . 68
4.2 Methodology . . . 69
4.3 Experiment Design . . . 69
4.4 Results . . . 69
4.5 Discussion . . . 69

Chapter 5: Conclusion **69**
5.1 Future Work . . . 69
5.2 Conclusion . . . 69

Bibliography **69**

List of Tables

List of Figures

1.1 A simple graph with three vertices and two edges. 19
1.2 A simple attributed graph with three named vertices and two edges. 20
1.3 A simple attributed graph with three named vertices and two directed edges. . . . 20
1.4 A simple attributed graph with three named vertices and two directed weighted edges. 20
1.5 A simple attributed graph with three named vertices and two directed labeled edges. 20
1.6 A heterogeneous directed attributed graph with three 'employee' vertices and two 'company' vertices. A very simple knowledge graph. 21
1.7 A heterogeneous directed attributed multi-graph with three 'employee' vertices and two 'company' vertices. Multiple different types of edges can exist between nodes of the same type. A very simple knowledge graph. 21
1.8 Example graph with five nodes and five edges 23
1.9 Adjacency List for graph in figure 1.8. Each line represents a vertex with the first in the list being the source and each subsequent vertex a destination of an edge. . 24
1.10 Adjacency Matrix for graph in figure 1.8 is a $jVj \times jVj$ matrix where a value of 1 indicates an edge and 0 is no edge. 24
1.11 Incidence Matrix for graph in figure 1.8 is a $jEj \times jVj$ matrix where a -1 is the source node and 1 is the destination of a given edge. 25
1.12 The Compressed Sparse Row (CSR) format uses 3 lists to represent the graph in figure 1.8. 25
1.13 A simplified representation of a direct graph in vertex-order where each vertex is added to memory in the order it was created (here, alphabetically). Edges are not represented here. 44
1.14 A simplified representation of a direct graph in vertex order where each vertex is added to memory in the order it was created with attributes stored separately. Pointers from topology to attributes not shown for clarity, but correspond to first letter of attribute. 46

3.1 Taxonomy of Benchmark Components 64

List of Abbreviations

AI	Artificial Intelligence
AIS	Automatic Identification System
ARLIS	Applied Research Laboratory for Intelligence and Security
BC	Betweenness Centrality
BFS	Breadth First Search
BLAS	Basic Linear Algebra Subprograms
BSP	Bulk-Synchronous Parallel programming model
C4ADS	Center for Advanced Defense Studies
CC	Connected Components
CCTV	Closed Circuit Television
CRUD	Create, Read, Update, Delete (database operations)
CSC	Compressed Sparse Column
CSP	Constraint Satisfaction Problem
CSR	Compressed Sparse Row
D3A	Decide Detect Deliver Assess (US Army targeting methodology)
DAG	Directed Acyclic Graph
DARPA	Defense Advanced Research Projects Agency
DCM	Data-Centric Model for programming
DoD	United States Department of Defense
DFS	Depth First Search
DRAM	Dynamic Random Access Memory
DSL	Domain Specific Languages
FLOPs	Floating Point Operations
FLOPS	Floating Point Operations per Second
FLOPS/W	Floating Point Operations per Second per Watt
GAP	Berkeley Graph Algorithm Platform
GAS	Gather-Apply-Scatter programming model
GCN	Graph Convolution Network
GNSS	Global Navigation Satellite System
GNN	Graph Neural Network
HIVE	Hierarchical Identify Verify Exploit
IIoT	Industrial Internet of Things
I/O	Input/Output
IoT	Internet of Things
IPC	Instructions per Cycle
I&S	Intelligence and Security

ISR	Intelligence, Surveillance and Reconnaissance
LLC	Last Level Cache
MIT	Massachusetts Institute of Technology
MISO	Military Information Support Operations
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
NMI	Normalized Mutual Information
NUMA	Non-Uniform Memory Access
OE	Operating Environment
OPSEC	Operational Security
OSINT	Open Source Intelligence
OSM	Open Street Maps
PAI	Publicly Available Information
PED	Processing, Exploitation and Dissemination
RAM	Random Access Memory
RSI	Remote Sensing Imagery
SGM	(isomorphic) Sub Graph Matching
SNAP	Stanford Network Analysis Project
SOTA	State of the Art
SPM	Spatial Pattern Matching
SSSP	Single Source Shortest Path
STGNN	Spatio-Temporal Graph Neural Network
TB	Terabytes
TEPS	Traversed Edges per Second
TEPS/W	Traversed Edges per Second per Watt
VIP	Very Important Person

Chapter 1

Introduction

Graph data structures and the algorithms that operate on them are foundational to computer science. Since the early days of the field, they have been a focal point for theoretical computer science. As the social media platforms of the early 21st century have proliferated into the global consciousness, the world has become increasingly aware of the descriptive power of graphs to model the real world. The Intelligence and Security (I&S) community uses graphs to structure collected information, analyze it and generate insights to drive operations in the national interest. However, the billions-to-trillions of edges scale the I&S deals in, the dynamic nature of graphs, temporal constraints on data possession and the criticality of their work create an imperative to improve the *efficency* and *effectiveness* of large-scale graph analysis.

Improvements can typically come from three sources. First, the *algorithms* used to analyze the graphs can be improved, or modified to operate in a time-constrained setting, for example by implementing early-stopping, incremental solving or sampling. Second, the underlying *data structure/implementation paradigm* can be modified to exploit new technologies, for example by moving to shared memory parallelism or switching from the canonical 'pointer and memory-block' representation to a linear algebraic representation. The third major avenue for improvement is using *hardware acceleration* to operate on graphs more *efficently* and *effectively*, for example by reducing network latency in distributed systems or reducing memory access latency.

This Thesis focuses on exploring *Hardare Acceleration* for graph processing. In particular, it explores if and when graph hardware accelerators should be used, and demonstrates how the

process of comparing across hardware architectures can generate insights about the problems themselves that can inform future algorithmic and implementation work.

1.1 Intelligence and Security Context

A good benchmark must reflect the reality of the problems that it is measuring, with conventional wisdom telling us that the best choice of benchmark to measure performance is real applications [1]. The *computational tasks* and the *data* that constitute the benchmark should be sampled from the target domain. In this section, an analysis of I&S use cases will reveal fundamental computational tasks that underpin graph analysis in the real world which inform the construction of the benchmark. A key point of difference from prior work is the focus on 'real-world' graph analysis problems rather than micro-benchmarking specific graph kernels. The real-world application is important, as it exposes a lurking difficulty of many graph analysis algorithms– they have a hybrid execution model where some parts are strongly memory-bound and others are compute-bound. A graph accelerator in isolation may improve the graph operations but sacrifice the 'conventional' computing performance to do so, resulting in a net loss overall. This section concludes by highlighting issues with the availability of datasets that continue to negatively impact research.

1.1.1 Use Cases for Intelligence and Security Problems

This section focuses on identifying the use cases for I&S problems drawn from an Applied Research Laboratory for Intelligence and Security (ARLIS) survey of datasets and transition opportunities [2] and a review of publicly available United States Military Doctrine [3–5]. The use

cases are deliberately drawn from the public domain and do not reflect any classified capabilities or research that may or may not be occurring. The purpose of reviewing the use cases is to extract intelligence and security problems that can be cast as graph problems and use them to prioritize the research effort.

The US Army Intelligence Doctrine identifies four key tasks [3]:

1. Supporting force generation,
2. Supporting situational understanding,
3. Conducting intelligence collection, and
4. Supporting targeting and information operations.

These four tasks provide a framework for use-case discovery. Each of the tasks has a collection of associated activities. The use cases have been collected by searching the commercial capabilities, open-source reports and academic literature for keywords associated with the tasks. From the review, exemplar use cases are selected to illustrate the intelligence problem it is addressing, approaches to the problems, the computational tasks underlying those problems and the expected impact on the analyst community should an optimal solution exist.

1.1.1.1 Supporting Force Generation

The focus of intelligence activities supporting force generation is to understand the likely operating environment of the force and develop contingency plans. While no relevant use cases exist within this task, it has an overall push for modernization and developing the systems to conduct intelligence operations that the use cases describe.

1.1.1.2 Supporting Situational Understanding

The focus of intelligence activities supporting situational understanding is on providing information about the relationships between friendly forces, adversarial forces and the operating environment. Typical activities include *Intelligence Preparation of the Battlespace* (IPB), *Developing estimates of adversary courses of action*, and *Supporting police and civil affairs*. Three use cases sit within *Supporting Situational Understanding*. *Population Segmentation Analysis* gives analysts a tool to understand the demographics of their operating environment. *Counter Threat Finance* activities support whole-of-government efforts to 'follow the money' and *Kill Chain Analysis* builds understanding about what indicators and warnings could provide early warning about future adversary activities.

Population Segmentation Analysis. A Military Information Support Operations (MISO) activity within IPB to divide a population into groups by geography, demography, attitude toward an issue or other thematic line. These groups or 'segments' are a subset of the population with common attributes to which a military force can tailor its communication [5].

The problem intelligence analysts must solve is determining who belongs to which segment of the population. Traditionally, the use of formal studies, surveys and interaction with the local population gained through sustained diplomatic and military interaction provided the data to segment a population. The modern era, however, gives analysts access to large volumes of publicly available social media and communication data. These datasets describe the interactions between people. A human social network. Prior work has shown how email patterns can predict social subgroups [6] at a small scale. A much larger project from the Massachusetts Institute of Technology's Sensable City Lab has shown that analyzing telephone calls in the United King-

dom yields strong structures in the communication graph that reflect population segments in the real world [7]. Social networks can be understood through their communication patterns, and the patterns of communication between individuals reveal insights about the structure of subcommunities, the patterns of information flow and who the most influential entities in the network are.

Information about the population can be represented as a graph. Each individual is a vertex, and each interaction is an edge. Weights may reflect the volume or durations of interactions, and attributes for vertices and edges may be used to incorporate additional information. The underlying computational problems here include *Graph Partitioning*, *Clique Detection* and more broadly *Community Detection*. Formally, the *Community Detection* problem involves the discovery of latent *Communities*, or "subgraphs whose members are more tightly connected than the broader graph" [8].

An *efficient* and *effective* community detection tool would yield immense benefit to intelligence operations, exploiting information that they already have to identify possible communities that they can use as start-points for their segmentation analysis, as part of their Intelligence Preperation of the Battlespace.

Counter Threat Finance. Is a collection of activities that occur across the intelligence and security community that have strong roots in civilian law enforcement. Counter Threat Finance (CTF) activities undermine threat finance–the means and methods used by an adversary to ensure their economic survival [9].

While CTF includes the disruption of threat finance networks by freezing, seizing or otherwise interrupting the flow of threat finance, our interest here is in the collection of financial intelligence to characterize and understand those networks. The primary source of financial

intelligence is raw financial data. Financial data generally consists of records of transactions, where money or assets are transferred from one entity to another. Traditionally, those transaction records transit some government or government-related banking system and access to these records through legal mechanisms can be exploited to search for malicious activity. Even financial systems that are not centrally controlled, like Bitcoin, can be analyzed to uncover nominally private information [10, 11]. Unfortunately, in either case, the malicious activity is hidden among the legitimate activity, and the job of the analyst is to determine which transactions and transaction patterns reflect malicious use. Prior work has shown that understanding the patterns of suspicious transactions that constitute threat finance enables analysts to search for similar patterns in new transaction data [12, 13].

Financial transactions can be modeled as a graph. Each account, or entity (depending on desired level of granularity) is a Vertex, and each transaction between two vertices forms an edge. The underlying computational problem here is *Isomorphic Subgraph Matching*, defined formally as given graph G and a query graph Q, retrieving all subgraphs of G that are isomorphic to Q [14]. That is, return all instances where Q is contained in G.

An efficient and effective isomorphic subgraph matching tool may enable financial intelligence analysts to pre-process enormous transaction datasets looking for transaction patterns that warrant detailed analysis, greatly increasing their capacity for counter-threat finance activities.

Kill Chain Analysis A kill chain analysis is a popular methodology for post hoc incident analysis that facilitates the learning required to improve readiness for future activities. The general idea is that for an incident to occur, an ordered set of events must precede it to set the conditions for the incident–the *kill chain*.

The canonical example draws from the counter-terrorism literature of the early 2000s,

which asserts that for a terrorist attack to occur an attacker must select the target, then reconnoiter it, select specific targets, surveil the target, rehearse the attack, conduct the attack and then exploit their successes and escape [15]. By understanding the chain of events, intelligence analysts seek to identify indicators and warnings from 'left of bang' that may indicate an attack is imminent. To prevent an attacker from exploiting their success, you would block their actions on the objective. To prevent the attack, you catch them in rehearsals. To stop them from rehearsing, you should catch them surveilling the target, and so on. Kill chains have also been popularised in the cyber domain by Lockheed Martin [16]. It follows the same idea that the goal of a cyber threat intelligence analyst should be to detect adversary activity long before they can act on their objective. The challenge facing the analysts is how to identify when these events, or strings of events are occurring as part of that kill chain, rather than being indicative of random noise. Prior work has shown that characterizing a kill chain as an attack graph (attributed multigraph that describes a computer network) can allow analysts to run what-if scenarios over their network to detect likely intrusion vectors [17]. They have also been characterized as knowledge graphs to support inferential reasoning about the paths an attacker could have taken through a network [18].

In a computing context, a kill chain can be represented as a graph. In the trivial case, the chain itself can form a Directed Acyclic Graph (DAG) and analysts would just map indicators of compromise to the known nodes in the chain. More complicated cases use a much richer representation and may encode the underlying structure of vulnerabilities, exploits, indicators of compromise and attacker goals [19]. The underlying computational problems here are *Graph Search* and *Shortest Path* aiming to determine if two nodes in the graph are *reachable*. That is, what paths exist between the two nodes, here is the attacker's start point, and their goal.

Understanding the paths available to an adversary enables an analyst to identify critical

nodes, enumerate critical paths, monitor for signs that those paths are being taken, and so empower the force to interdict the adversarial activity 'left of bang' in the future. Being able to build and search these large graphs of complex networks as part of incident analysis or simulation to answer what-if questions can help analysts decide where to target scarce assets for collection activities.

1.1.1.3 Conducting Intelligence Collection

The focus on intelligence activities supporting information collection is to synchronize and integrate the employment of sensors and assets to support current and future operations. That is, these activities support forces actively engaged in fighting. Activities may draw on earlier work in supporting situation understanding. Typical activities include *identifying high-value targets*, *processing collected information* and *generating forecasts of future adversary activity*. Three use cases illustrate problems typical of conducting intelligence collection. In *All-source fusion* analysts filter collected data for duplicates, contradictions and patterns–developing the informational blocks that become actionable intelligence. *Link analysis* helps analysts discover high-value targets and *intelligence assessment* activities develop forecasts for future adversary activities by detecting anomalies and predicting expected behavior.

All-Source Fusion All-source intelligence is the integration of intelligence and information from all relevant sources to analyze situations or conditions that impact operations [3]. All-source fusion is the task of integrating disparate sources, by consolidating, combining and correlating information sources [3]. That is, *processing collected information*, sometimes called the *Processing, Exploitation* and *Dissemination* (PED) process.

Identity Activities are a critical task within all-source fusion that requires analysts to synchronize the outputs of many disjoint single-source intelligence capabilities to discover the true identity of a threat actor–linking those identities to events, locations and networks. Determining which single-source observations are referring to the same entity is a grueling task, one that is made more difficult when access to the information is temporally bound [2] because of technical limitations or policy constraints. The problem applies more generally to event resolution. Prior work has shown the vital role of deduplicating events and resolving entities in knowledge graphs to combat human trafficking [20].

The all-source fusion problem can be cast as a graph problem in a few different ways. One approach is to make each reference in the data into a vertex and generate edges based on the similarity of those vertices. Other approaches reify the references to the entities in the graph as vertices. Given these representations *graph clustering* [21], *graph embedding* [22], *neural* [23], and *graph-theoretic approaches* that exploit topological and attribute information [24] have all been used on static graphs. Work on dynamic graphs uses an *incremental partitioning* algorithm to re-evaluate prior resolutions as new information is added to the graph [25].

Reducing redundancy in intelligence reporting and being able to effectively resolve all of the collected information back to the real-world entities that they relate to at a large scale gives analysts the ability to report more accurately and make assessments grounded in reality without having to manually verify and deduplicate every report.

Link Analysis A link analysis problem is a method of *processing collected information* requiring an intelligence analyst to determine if two entities, events or other phenomena are related by determining the strongest association paths between them [26].

The intelligence problem is frequently a question of social contact, an analyst will build a

list of known associates for each entity in question and expand those lists outwards until identifying a connection between the two. The problem can be multidimensional. A friends list on a social media platform may give one subset of connections, call data logs another, and the locations that emerge from physical surveillance of their pattern of life may form a third. Archetypically solved with a cork-board, pictures and red twine, more recently applications like the Analyst's Notebook[1] are used to create databases to represent each collected piece of information. Asking *"Does this entity associate with that entity"* is then a problem of following the twine to see where (if at all) the two are connected.

The photos and connecting twine are a physical representation of a graph. Here, the computational representation of the photo is a vertex, and each confirmed interaction or association described by the twine becomes an edge. The underlying computational problems here are *graph search*, *reachability* and *shortest path*. Each of these approaches can answer the question *"Does this entity associate with that entity"* slightly differently.

By representing collected intelligence as a graph, and exploring how they are related computationally, analysts can massively expand the scale of their analysis. There is, however, a gap–much intelligence work is based on assumption, assessment and hypothesis. How does the graph representation help the analyst in their hypothesis testing?

Intelligence Assessment. Kent: Source formal definitions, maybe split into anomaly detection and link prediction to make this shorter

Making assessments is core to intelligence work. *Information* only becomes *intelligence* with the addition of context and assessment. Generating assessments is a core task within *geneating forecasts of future adversary activity*. Perfect assessments require perfect information. However, intelligence analysts only get access to an incomplete and noisy view of the world. Collection

[1] `https://i2group.com/i2-analysts-notebook`

assets compete against adversary disinformation techniques which yield incomplete information and misinformation tactics that introduce noise. The assessment challenge is to identify where the true gaps in collected information are, filter out the noise and use the remaining information to develop forecasts about what the adversary is doing.

Towards *'reducing the noise'*, recent work theorizes that various Graph Neural Networks (GNN) trained on a baseline of 'normal' behavior of Industrial Internet of Things (IIOT) devices could detect *point*, *contextual* and *collective* anomalies in power grids, traffic flow and smart factory performance [27]. More concretely, GNNs have been used to detect anomalous behavior in telephone networks to identify scam calls [28].

Towards working with *incomplete information*, understanding the descriptive power of *Weak Ties* in a network is well known [29]. Some predictive power can be gained by applying naive approaches like triadic closure, focal closure, and link-centric preferential attachment to social networks, exploiting the observed behaviors of *weak ties*. Recent work from the Massachusetts Institute of Technology (MIT) Sensable Cities Lab shows how modeling information flows (email) of academics during the COVID-19 pandemic reflects the observed physical disruptions to co-location and the resultant impact on forming new contacts and developing 'weak ties' [30]. More specific to I&S problems, recent work has attempted to attribute terrorist attacks of unknown origin to their perpetrators using GNNs [31] and identify possible links between known criminals and new cases [32]. While these studies lack some scientific rigor, they highlight how the more general problem of link prediction for knowledge graph completion can be used in an intelligence context.

The computer representation echoes the representation needed by the all-source-fusion and link-analysis problems. The common term for these formats is a *Knowledge Graph*. A *Knowl-*

edge Graph is a structured representation of information, comprised of entities and semantic relations. Entities can be real-world objects and abstract concepts. Knowledge graphs are typically encoded as attributed graphs, consisting of 'facts' that are composed of a $\{$*Head, Relation, Tail*$\}$ triple where the head and tail are entities and the relation is the semantic description of their connection [33]. Knowledge graphs quickly become very large and have heterogeneous vertex and edge types, which complicates graph processing. The anomaly detection work involves *training* a GNN and *inferencing* on unseen data to detect deviations from the norm. The link prediction to identify gaps involves similar training and inference operations or the creation of embeddings from the topological and attribute features of the knowledge graph.

Most of the surveyed problems see data represented in a knowledge graph, and in real terms, these are the input data structures to the intelligence programs described in this section. Being able to detect anomalies to 'de-noise' the data analysts are reviewing or make support forecasting and assessments by 'filling in the gaps' at a large scale can massively boost the reach and productivity of analysts and improve the yield of the other analytic techniques discussed in section 1.1.

1.1.1.4 Supporting Targeting and Information Operations

Targeting is the process of selecting and prioritizing targets and matching the appropriate response to them, considering operational requirements and capabilities. There are numerous targeting methodologies. For example, the US Army uses the decide, detect, deliver, and assess (D3A) methodology to enable tactical commanders to engage targets within their operating environment (OE) [4]. Intelligence activities support every step in the targeting process. The *decision*

is largely driven by the IPB, *detection* and *assessment* are enabled by intelligence collection and processing activities. Typical activities include *target acquisition*, *target tracking* and *target exploitation*. The two exemplar use cases from targeting are heavily couched in the physical world. *Geospatial Analysis* is how analysts acquire targets, track their position in the real world and support the delivery of an effect. *Localization* is a sub-problem within geospatial analysis that challenges analysts to identify 'where in the world' a target location is.

Geospatial Analysis. Kent: Need to link to the doctrine

Geospatial analysis is an important tool used by analysts. Typically, the adversary is operating in the physical world, with limited time and assets and so by tracking where they are, and when, an analyst can drastically reduce the uncertainty surrounding their actions. Spatio-temporal data is widely available and heavily researched. For example, the Centre for Advanced Defense Studies (C4ADS) used data from the maritime Automatic Identification System (AIS) to detect the use of the Global Navigation Satellite System (GNSS) spoofers and correlate it with Russian Very Important Person (VIP) protection activities [34]. Spatio-temporal analysis has also been used to detect when ships are on collision trajectories [35] and contact tracing in epidemiology [36, 37].

Information about the movement of entities can be acquired through Closed Circuit Television (CCTV) cameras [38], physical or technical surveillance and by accessing the location data collected by smart devices. Given a small amount of data about locations, times, identities and a model of who is being collected against, it is trivial to determine when and where individuals associate [39]. Discovering the behaviors of groups is much more complicated, and significant research effort into detecting and tracking *flocks, convoys, swarms, groups* and *gatherings* through group pattern mining is ongoing [40]. Direct intelligence applications include determining the associates of known adversaries, determining patterns of life, identifying key locations and enu-

merating a threat network. At the micro-level, this is known as the 'co-movement' or 'co-traveler' problem. The co-traveler problem involves discovering when two or more entities have been spatially and temporally co-located [2]. While not explicitly a targeting problem, it is an effective method to *find* new targets of interest, and supports the *decide* and *detection* phases of targeting.

Most of the computational approaches use a multi-stage approach to the co-traveler problem. First, they extract the spatial-temporal data for each entity of interest, extracting the position of each entity at each timestep. Second, they break the dataset up by timestep. So, a dataset of T timesteps becomes T datasets of a single timestep. Next, a density-based clustering algorithm like DBSCAN [41] associates entities into groups at each timestep. Finally, looking through all timesteps, entities that are in the same group for some time-based threshold are said to be 'co-travelers ' [40]. Most of the reviewed approaches do not use graph-based approaches. Those that do tend to use them as an alternative representation to the clusters. They use proximity graphs at each timestep where the vertices are the entities, and the edges are formed between entities that have come within some proximity threshold during the timestep. Three exceptions include:

1. Grounding the co-traveler analysis in a graph representation of the road network [42],
2. Building a Movement Companion Linkage Graph (MCLG) using Automatic Number Plate Recognition (ANPR) cameras [43] and,
3. Using Maximal Clique detection and Maximal Connected Subgraph discovery in a connectivity graph to model flock and convoy behavior [40].

The availability of spatiotemporal data and the power it offers to ground intelligence in the physical world, combined with the opportunities it offers for discovering connections for targeting highlight the utility to the I&S community. Of the reviewed co-traveler analysis approaches

the use of graph data structures and algorithms was uncommon, and so co-traveller analysis may not be a suitable benchmarking problem.

Localization Kent: Need to link to the problem Localization is a technique within imagery analysis where an analyst must find the location on a map that a particular photo, description or other report is referring to.

Localization arises frequently in support of targeting. Some partial information about a target of interest is received, and the analyst must determine where that target is. Often materializing as a manual comparison to collected Remote Sensing Imagery (RSI) from aircraft or satellite surveillance, analysts spend significant amounts of time trying to locate where exactly their target is. Computer vision approaches struggle with object detection and pattern matching in RSI because of the directional invariance, loss of detail and lack of defining features when looking 'down' on things [44–47]. Alternate approaches that do not use RSI have looked at matching sketch maps to real maps [48], building geotagged databases from ground-level imagery and Open Street Maps (OSM) data [49], or casting the world as a collection of points of interest and the distances between them [50]. While none of these papers dealt explicitly with an I&S use-case, they do show how the localization process can be augmented with computing.

The underlying computational problem in localization is *Spatial Pattern Matching* (SPM). SPM uses topological, metric or directional constraints between 'objects' or 'locations' to represent the real world and issue queries. Because of the 'object' and 'constraint' structure, SPM databases are almost ubiquitously represented as attributed graphs, where the vertices are the objects and the edges represent the constraints between each vertex. As a result, these are almost invariably approached as an Isomorphic Subgraph Matching (SGM) task or Constraint Satisfaction Problem (CSP). As CSP and SGM are NP-complete problems, heuristic approaches are used

to approximate solutions [51].

SPM is a valuable technique that will greatly assist analysts in the task of localization but is constrained by the slow processing of graphs with heuristic algorithms that rely heavily on graph operations.

1.1.1.5 Summary of Intelligence and Security Problems

These eight use cases are a representative sample of the problems faced by the I&S community and share common threads. First, all of the use cases are difficult to scale in the physical world. There is only a finite amount of twine to 'join the dots', and analysts can only retain so much information while they are working. Second, the focus is on characterizing the relationships between entities, events and objects, and using these relationships as the basis for further analysis. Third is the need to complete these tasks within time, resource and information constraints. Lastly is the realization that the key benefit is to replace the 'procedural' aspects of intelligence work so that the analysts can focus on generating assessments rather than the tools to clean and structure data.

The underlying computational problems will be explored in greater detail in section 1.2, but as they form half of the drive for the benchmark key observations from the use cases are noted here. First, the underlying representation of almost all the I&S problems is an attributed graph or multigraph. Typically, these are *Knowledge Graphs* that contain heterogeneous vertex and edge types. Almost all of the computational problems rely on the structure of graphs to generate their results–whether traversing along edges to find links, enumerating the neighborhood of a vertex of interest or transforming the topology and attribute properties into an embedding for further

analysis. The nature of the problems being solved is only half of the problem that a benchmark needs to consider.

1.1.2 Intelligence and Security Datasets

Datasets that are from the same distribution as the problem domain ensure that a benchmark is measuring behavior that we expect in the real world. An obvious challenge is that the I&S community's data is at best likely to contain Personally Identifiable Information (PII) and at worst will be classified. In either event, raw domain data are unlikely to be available for a public benchmark because of privacy and security concerns. The lack of 'real' data is particularly impactful in graph processing, where the efficiency and effectiveness of the algorithm is highly dependent on the shape of the input graphs. Kent: Need to cite this, probably beamer's work .

Privacy preservation is an area of active research across many domains. In graph analysis for the I&S domain, privacy preservation would extend to include protecting classified information. To create a synthetic graph that reflects the heterogeneous attributes and topologies of graphs of interest to the I&S community, the ideal solution would be to extract a statistical description of the original graph that would permit a generator to create a similar, but completely disjoint graph. Approaches to realistic graph generation work using either an adjacency-based or statistically-based input, leveraging parametric or neural methods to generate new graphs from the input. Some of the most recent work on generating realistic graphs, GenStat uses the input features of *Triangle Histograms, K-Hop Neighbour Histograms [k=1..4], Random Walk Histograms [r=2,3]* and *Number of Edges*. They show that they can produce topologically similar graphs using these four statistical features as input to a graph generator that uses a probabilistic

model of graph statistics implemented using a variational auto-encoder [52]. The first of two key limitations of this approach is the loss of attribute information during the encoding process, which is key to meaningful analysis of the graphs. The second is the scalability–the complexity of generating the descriptive statistics is cubic in the number of edges. Their experiments were on graphs with just thousands of vertices, and this approach (along with many others) will not scale to the billions-to-trillions of edges expected in I&S Problems.

The next best method is to find datasets that are reasonably expected to resemble I&S problem sets from repositories like the Stanford Network Analysis Project (SNAP) [53], the Berkeley Graph Algorithm Platform (GAP) [54] or using generators that can create graphs with enough control to generate interesting problems.

1.2 Graph Data Structures

Kent: @REVIEWERS - START HERE FOR NOW

Kent: Add section overview

1.2.1 Notation

This section outlines the notation and terminology that will be used throughout the remainder of this thesis. A *Graph*, sometimes called a *Network* in the social sciences, is a foundational data structure and computing concept to describe combinatorial structures [55]. The simplest graph G has two components. The *Vertices* V, sometimes called *Nodes* is a set of objects that represent 'entities' or 'things'. The *Edges* E, sometimes called *Arcs* or *Links* are the set of connections between the vertices that describe some relation between them. So, the graph G has the

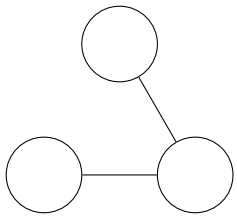


Figure 1.1: A simple graph with three vertices and two edges.

sets of vertices (V) and edges (E), giving us $G = fV; Eg$. The *Degree* (K) of a vertex is the count of the edges it participates in. In directed graphs, it is often seperated to *in-degree* (K_{in}) and *out-degree* (K_{out}). The structure of the graph, the pattern formed by the vertices and edges that connect them is the *topology* of the graph. The *topology* is the distinctive feature of a graph data structure. Analyzing and exploiting the *topology* is core to many algorithms that seek to solve graph problems. The *diameter* of the graph is the length of the shortest path between the two most 'distant' nodes. The *eccentricity* of a graph is the greatest geodesic distance between two vertices in the graph [56]. The *radius* of the graph is the minimum graph eccentricity. [56] The *algabreic connectivity* of a graph describes how well-connected it is [56] the *traversal depth* of a graph algorithm is how many traverse iterations need to occur to complete an algorithm and is directly proportional to the eccentricity and connectivity of a graph [56].

Figures 1.1– 1.7 show how quickly complexity builds in graphs with the addition of attributes (labels), direction, weights and heterogeneous vertex and edge types.

Kent: Combine into subfigures

1.2.2 Graph Representation

Graphs can be represented in numerous ways, with the two most common being the *Adjacency List* and *Adjacency Matrix* formats. Consider the graph in figure 1.8 with five vertices and

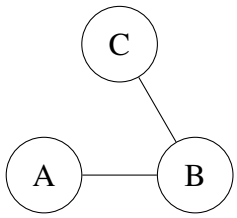


Figure 1.2: A simple attributed graph with three named vertices and two edges.

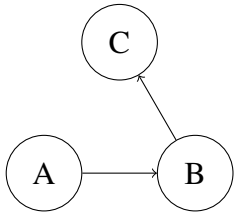


Figure 1.3: A simple attributed graph with three named vertices and two directed edges.

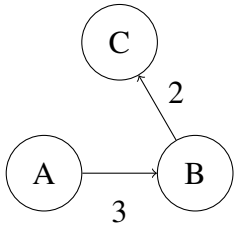


Figure 1.4: A simple attributed graph with three named vertices and two directed weighted edges.

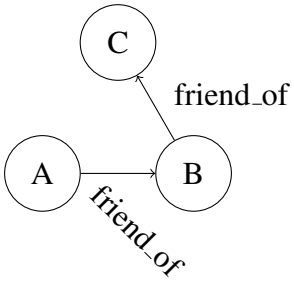


Figure 1.5: A simple attributed graph with three named vertices and two directed labeled edges.

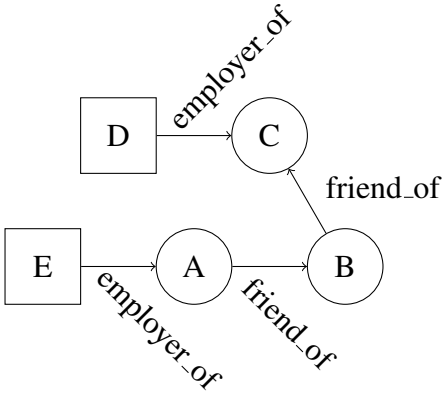


Figure 1.6: A heterogeneous directed attributed graph with three ‘employee’ vertices and two ‘company’ vertices. A very simple knowledge graph.

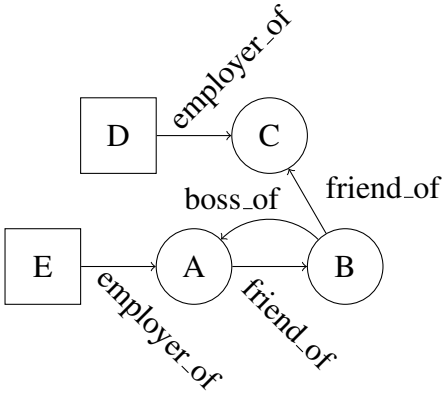


Figure 1.7: A heterogeneous directed attributed multi-graph with three ‘employee’ vertices and two ‘company’ vertices. Multiple different types of edges can exist between nodes of the same type. A very simple knowledge graph.

five edges. Representing it using an adjacency list (figure 1.9) yields a small memory footprint ($O(|V| + |E|)$), but finding a particular edge is expensive being linear in the number of vertices ($O(|V|)$) in the worst case. Conversely, representing it as an *adjacency matrix* (figure 1.10) has a memory footprint of $O(|V|^2)$ but a constant lookup time ($O(1)$). The tradeoff between memory efficiency and lookup efficiency is a core tension in graph processing and many of the attempts to accelerate the efficiency of graph algorithms focus on balancing these two dimensions (see section 1.3).

The adjacency list and adjacency matrix are capable of implicitly encoding the direction of edges but are not innately capable of representing edge weights or multigraph structures. Another encoding of graphs, the *incidence matrix*, encodes both the source and destination of an edge. The representation in figure 1.11 uses a negative integer to indicate the source, and a positive to indicate the destination. The integers can be replaced with any real number to represent weights, encoding more information. Because the edges are represented as distinct entities, the *incidence matrix* is capable of representing more complicated graph formats that are directed, weighted, multi-edge and hyper-edge [57]. The space complexity of an incidence matrix is $O(|V| \times |E|)$ and so for sparse graphs may be more space-efficient than an *adjacency matrix* but in general will consume more memory where $|E| > |V|$. The lookup time for edge existence in an incidence matrix is also slower than for an adjacency list. To find an edge connecting two vertices, we need to scan the entire edge list for each vertex and locate which edge(s), if any, are incident to both vertices, reducing to a complexity of $O(|E|)$.

Many formats have been developed to improve the balance between storage efficiency and lookup speed. The dominant choices are the Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC) formats. The CSR format uses three lists to compress the rows of a sparse

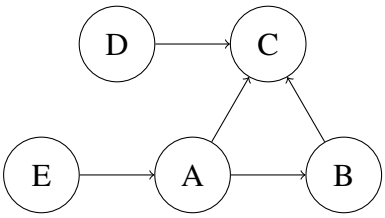


Figure 1.8: Example graph with five nodes and five edges

matrix, like an adjacency matrix [58]. First, the *value list* contains every nonzero value of the matrix (the existence of an edge), and so has a length of $|E|$ in a directed adjacency matrix. The *Row Pointer* list contains the index of each position in the *value list* that is associated with a new row. The *Column Indices* list contains the column index of each value in the *value list* from the original sparse matrix. So in figure 1.12, the values associated with the **A** vertex (row 0 of the adjacency matrix) are at positions $[0; 1]$ in the value list in figure 1.12. We derive this by looking at the row index in figure 1.12 and seeing that the second row begins from index 2. Knowing that the first two values in the value list are the sources of our edges, we know that the first two values in the column index in figure 1.12 are the destinations $[1; 2]$. In the adjacency matrix (figure 1.10), column 1 is for vertex **B** and column 2 is for vertex **C**.

The CSC format reverses the function of the row and column indices so that the columns are compressed. The space efficiency for CSR/CSC format is $O(|V| \log(nnz) + nnz \log(|V|))$ for an adjacency matrix of $|V|$ vertices [58] and takes $O(\log(nnz))$ time to check the existence of an edge where nnz is the count of non zero entries in the matrix.

More complicated representations exist which are capable of compressing the graph representations even further. *Ligra+* uses an encoding scheme that splits the graph into two arrays *Compressed Edges-in* and *Compressed Edges-out*. The arrays use k-bit block encoding to approximately halve the memory requirements to represent each graph compared to CSC/CSR. The

$A;$ $B;$ C

$B;$ $C;$

$C;$

$D;$ $C;$

$E;$ $A;$

Figure 1.9: Adjacency List for graph in figure 1.8. Each line represents a vertex with the first in the list being the source and each subsequent vertex a destination of an edge.

	A	B	C	D	E
A	0	1	1	0	0
B	0	0	1	0	0
C	0	0	0	0	0
D	0	0	1	0	0
E	1	0	0	0	0

Figure 1.10: Adjacency Matrix for graph in figure 1.8 is a $jVj \times jVj$ matrix where a value of 1 indicates an edge and 0 is no edge.

$$\begin{array}{c|ccccc} & e_0 & e_1 & e_2 & e_3 & e_4 \\ A & -1 & 0 & 0 & 1 & -1 \\ B & 1 & -1 & 0 & 0 & 0 \\ C & 0 & 1 & 1 & 0 & 1 \\ D & 0 & 0 & -1 & 0 & 0 \\ E & 0 & 0 & 0 & -1 & 1 \end{array}$$

Figure 1.11: Incidence Matrix for graph in figure 1.8 is a $|E| \times |V|$ matrix where a -1 is the source node and 1 is the destination of a given edge.

$$\begin{array}{rcccccc} \text{Value List} & 1 & 1 & 1 & 1 & 1 & \\ \text{Row Index} & 0 & 2 & 2 & 3 & 4 & 5 \\ \text{Col Index} & 1 & 2 & 2 & 2 & 0 & \end{array}$$

Figure 1.12: The Compressed Sparse Row (CSR) format uses 3 lists to represent the graph in figure 1.8.

compression has an encode/decode overhead that makes it impractical in serial systems but has been shown to improve overall performance in parallel implementations [59].

Graphs can be *Static* or *Dynamic*. Static graphs are those that are pre-built and unchanging. Algorithms can assume unlimited time to find a solution and that the representation itself will remain unchanged. A *Dynamic* graph (also called a *streaming* graph is constructed on the fly, with vertices and edges being created, read, updated and deleted (CRUD) while graph analysis is occurring. The I&S community is constantly collecting and removing data, and so must be able to operate on graphs in time-constrained settings between when information is added and when it is removed [2].

Kent: Summary of Representation here

1.2.3 Graph Primitives

Kent: The back half of the primitives section is just points I collected in my travels and need to synthesize

There is no real agreement on what the building blocks of graph processing should be [60]. The *adjacency list, adjacency matrix, incidence matrix* and *CSC/CSR* formats are all used for *canonical* representations of graphs. The *canonical* representation of a graph views the vertices as a chunk of memory, and the edges as pointers between those chunks. These chunks may also include details of the labels, weights and other non-topological information stored in the graph, or just house pointers to that information. Using this representation for large and dense graphs causes the 'chunks' of memory for neighboring nodes to not be spatially co-located. Without spatial locality, modern processor optimizations like caching and prefetching are ineffective, and the processor will need to wait for the relevant node vertex to be retrieved from the main memory

before continuing.

Kent: Add in Combinatorial BLAS here. Drawing inspiration from the relationship between the Basic Linear Algebra Subprogram (BLAS) library and the development of accelerators for Floating Point Operations (FLOPs), an alternative linear algebraic representation of graphs and implementation of graph algorithms is emerging. The GraphBLAS project is an effort to formalize graph operations using linear algebra [61]. Using the incidence matrices for graphs in a CSC/CSR format, the GrahBLAS is building a mathematical abstraction layer for graph operations based on the realization that graph operations are matrix operations [57, 62].

The GraphBLAS team identified five primitive operations and five additional compound operations that can be composed of primitive operations. The five primitive operations are *Construct Matrix from Triples (**Sparse**)*, *Extract Triples from a Matrix (**Find**)*, *Element-Wise addition (**SpEWiseA**)*, *Element-Wise Multiplication (**SpEWiseX**)*, and *Matrix Products (**SpGEMM**)* [62]. These primitive operations are composed to represent graph operations, like a single step in a Breadth First Search (BFS) which is a matrix-vector multiplication of the adjacency matrix and the vector representing the node(s) to start from [57].

Kent: Here down is 'collection of things I should probably include but haven't integrated yet' Lots of it is redundant and covered later.

Wu et al identify two families of graph primitive operation, *traversal*-based and *dense-computation*-based. Traversal-based primitives systematically explore neighbors until some condition is met and only deal with small subsets of vertices. Dense-computation primitives expect most or all vertices to be active. Traversal primitives include BFS, Betweenness-Centrality (BC), Single-Source Shortest Path (SSSP) and depending on implemenation Connected Components (CC) [56]. Dense-computation primitives include PageRank [56]. Ham includes *Collaborative*

Filtering as a dense-computation primitive [63]. Each of these 'primitives' is an algorithm in its own right, and are commonly thought of as *kernels* rather than a 'primitive' operation.

Some programming models offer more conceptual primitives. GPU-based programming models utilize the Bulk-Synchronous Parallel (BSP) programming model. The *Gather-Apply-Scatter* (GAS) is a BSP approach that decomposes a graph program into the *gather* phase which accumulates information about all the adjacent vertices. The *Apply* phase creates a new attribute on the active vertex with the computed results of the accumulated values from the previous step. The *Scatter* phase propagates a predicate evaluation on all adjacent edges and vertices [56]. The *Data-Centric Model* (DCM) uses the *Advance, Filter* and *Compute* operations to manipulate the *frontier* of vertices. The *frontier* is the collection of vertices currently 'active' in the algorithm. The *Advance* operation generates a new frontier by expanding the neighbors of the current vertex frontier. the *Filter* operation chooses a subset of the frontier based on some specified criteria. the *Compute* operation executes some specified action on all vertices in the frontier [56].

Several fundamental graph algorithms cannot be expressed as frontier-based algorithms. [64]

The Pegasus graph analysis system also takes a linear-algebraic view. It introduces the *Generalized Integrated Matrix-Vector Multiplication* (GIM-V), a generalization of standard matrix-vector multiplication. They decompose it into three atomic operations *combine2, combineAll* and *Apply*. They use these three operations to implement *PageRank, Random Walk, Diamater Estimation* and *Connected Components*.

There are two families of Programming Models for Graphs. *Vertex-Centric* and *Edge-Centric*. *Vertex-Centric* models effectively express the entire algorithm as operations on a single vertex and its edges [63] *Vertex-Centric* models have two functions *vertex_scatter* and *vertex_gather*. *Vertex_scatter* defines what operations need to be performed on a vertex, and *ver-*

tex_gather applies that operation and updates the vertex. The *Edge-Centric* model has two functions *edge_scatter* and *edge_gather*. *Edge_scatter* sends the information of the source vertex and edge to the destination vertex where the *edge_gather* function applies the updates [65].

The *Single Instruction Multiple Data - X* (SIMD-X) paradigm defines *Active, Compute* and *Combining* operations. Is a BSP paradigm that aims to extend beyond vertex or edge centric computing. The *Active* function filters the graph and returns only the vertices which meet the conditions to be 'active'. The *Compute* function defines the computation that happens on each edge. *Combine* merges all the computed updates.

The 'canonical' representations do not appear to have clearly defined primitive operations in the general sense, though high-performance parallel processing programming paradigms have done significant work in this area [66].

The reasons that distilling primitive operations is important in benchmark design are, first, that identifying the primitives allows us to design micro-benchmarks to understand how each building block of an algorithm can be optimized. Second, by limiting to a set of primitives, the hardware implementation of those primitives can be optimized, meaning that the hardware designers can focus on developing hardware for a few specific operations, rather than general-purpose ones.

1.2.4 General Characteristics

Decades of research yield the characteristics of graphs that are 'generally true' and interact with the *efficency* of graph analytics. These graph characteristics focus on the *topological* features of graphs. Depending on the context, the addition of attribute information may simplify or

complicate the algorithms that operate on the graphs.

Input dependence. The *efficiency* and *effectiveness* of every graph algorithm depends on the size and shape of the input graph. The data dependence of graph algorithms is one of the driving reasons that empirical evaluation favors a collection of 'real-world' graphs drawn from the domain of intended use. While the shapes of graphs exist on a spectrum, common 'classes' of graphs exist. *Random Graphs* are the baseline. A random graph is synthetic. It is generated by creating a set of vertices and then creating an edge between each vertex in the set with a given probability. Formally, given $G(n; p)$ where n is the number of vertices to create and p is the probability of any two vertices being connected by an edge, the generator creates n vertices $fv\ 2\ Vg$, and the set of edges $E\ =\ f(u; v)ju; v\ 2\ V; u\ 6=\ v$ with probability Pg yielding the random graph $G\ =\ fV; Eg$ [67]. Random graphs are important as they provide the point of reference for other graph characteristics, discussed below.

Size Matters. Section 1.1 identifies that graphs in the I&S domain can be billions to trillions of vertices and edges. The *size* of a graph impacts graph processing. In the most obvious sense, like all computations, there is some relationship between the size of the input and the time required to process it. The count of the vertices can give us a false understanding of the scale of the graph. For example, suppose a graph with 10 vertices. Each vertex except for the first and last is connected by an undirected edge to only two other vertices, such that they form a straight line of 10 vertices. The graph will have 10 vertices and 9 edges. But, suppose a fully connected graph, where each of those vertices is connected to every other vertex. That then becomes a graph of 10 vertices and 45 edges. If directed (and symmetric), we would have 18 and 90 edges respectively for the same graph of 10 vertices. If it is a more complicated multi-graph structure the edge count could increase even further. So, any discussion of graph 'size' must also account for the number

of edges. Kent: Needs a Cite

Degree Distribution. The *size* of the graph is closely linked to the number of edges in it. Beyond a raw edge count, consideration must be given to the Degree K, the mean degree $\bar{K}$, the mean in-degree $\bar{K}_{in}$ and mean out-degree $\bar{K}_{out}$ and the *distribution* of these values across the graph. Random graphs are quite regular in their distribution of degrees, and we expect the modal degree to approximate the mean degree. Similarly, on graphs derived from physical phenomena like road networks, the mean degree is generally consistent. Conversely, graphs derived from social interactions like phone call records or follower lists on social media tend to follow a power-law distribution. In these *'Scale-Free'* graphs only a small percentage of the vertices possess most of the edges, and most vertices have very few edges. Intuitively, consider that a few celebrities are extremely popular and have large followings, while most people have only small followings. The scale-free graph shape is one of the key drivers of computational *inefficeny* in graph processing. (see: section 1.2.6). Kent: Needs a Cite

Graph Diameter and Transitivity The *diameter* of the graph is the length of the shortest path between the two most 'distant' nodes. The *transitivity* describes the probability that adjacent nodes in a graph are connected. The *clustering coefficient* is a measure of the transitivity at a local or global level (with slightly different calculations depending on scope).

Random graphs tend to have a middling diameter and clustering coefficient, depending on the density parameter used to generate the graph. Graphs derived from physical phenomena like road networks tend to have a large diameter and small global clustering coefficient. Scale-free graphs representing social phenomena like social media follower graphs tend to have a small diameter and large clustering coefficient. Known as the *small-world* characteristic, we can say that of a scale-free graph, most vertices are 'close' to each other. Some graphs have a *small-world*

property in some component subgraphs but have other subgraphs that are of a larger diameter and lower density.

The variance in the diameter and density of graphs, even internally on large complicated graphs like *knowledge graphs* means that a graph processing algorithm may need to intermittently deal with extremely dense and sparse sections within the same algorithm, complicating the ability to efficiently process them. Kent: Needs a Cite

Community Structure. Closely linked to the idea of degree distribution and clustering coefficient is *community structure*. Characterized by Michelle Girvan Mark and Newman in 2002, a community is broadly defined as a subgraph that is more densely connected to itself than it is to the remainder of the parent graph [68]. It is a generalization of *Cliques*, which are fully connected subgraphs within a parent graph. Communities are more distinct in graphs that have variance in their degree distributions and local versus global structure. Exploiting the latent information in communities relies on the variance in graph structure and provides a useful highlight of why graphs of heterogeneous shape are not only expected but essential in many of the graph analysis tasks.

Sparsity. The discussion of size, degree, diameter, transitivity and community structure highlights that it is the *topology* of a graph that contains much of the latent information that graph analysis algorithms try to extract. A defining feature of graphs is their *sparsity*. *Sparsity* is most easily understood by looking at an adjacency matrix (figure 1.10). Of the $|V|^2$ matrix, only $|E|$ of the values contain 'information', with the remainder of the values being 0. There are two impacts relevant to graph hardware acceleration. First, if the graph is stored as an adjacency matrix, it also needs to allocate memory to represent the 0s, greatly increasing the memory footprint. Second, if the graph is stored in the canonical format, even if the 0s are not represented,

many memory management approaches will create the graphs in memory in the order that the vertexes are created, potentially leading to vertices connected by edges (neighbors) being stored in different locations in memory, and so will not be pre-fetched together when being processed. More sophisticated graph database systems use the *index-free adjacency principle* to colocate adjacent nodes in memory and use internal pointers to track the locations of neighbors without a separate index [69].

1.2.4.1 Summary of Graph Characteristics

Kent: Summary here

1.2.5 The Intelligence and Security Graph Problems

1.2.5.1 Problem Characteristics

Kent: Need to reduce this down to more foundational problems. These are important as they will form the basis of the problems in the benchmark. NOTE that I need to update this to reflect the graph-based AI language after I update chapter 1.

Kent: @Nicole - Skip this section for now, I need to do some heavy restructuring.

Section 1.1 identified eight use cases that apply to the I&S community that represent intelligence problems as a network (graph). Those eight use cases are addressed with seven distinct graph computing tasks. This section briefly reviews these tasks and known challenges to identify the impacts on graph processing.

Community Detection. The *population segmentation* intelligence task can be character-

ized as a community detection problem in graph analytics. The community detection problem was formalized by Girvan and Newman [68] and has become a mainstay of challenging graph processing problems. It has been variously defined as '[the task of detecting] *a subgraph whose members are more tightly connected than the broader graph*' [8] and '[the task of detecting] *cohesive subgroups in networks*' [70]. While various approaches exist, *Modularity Optimization* dominates. *Modularity* is a measure of how well connected a graph is compared to a random graph. The task of optimizing modularity requires determining the optimal partitioning of a graph so that each of the partitions has its maximal possible modularity [71]. Unfortunately, modularity optimization has been proven strongly NP-Complete [72], so heuristic approaches must be employed unless $P = NP$. Kent: Common computations?

The community detection problem can be simplified with the addition of external information like known community membership constraints on a subset of the vertices in the graph.

Isomorphic Subgraph Matching Both the *Counter Threat Finance*, *Spatial Pattern Matching* and some implementations of *Co-Traveler Discovery* intelligence problems can be characterized as isomorphic subgraph matching (SGM) problems. The graph isomorphism problem asks whether a query graph Q and a reference graph G are topologically identical [73]. The subgraph isomorphism problem is a special, more difficult case of graph isomorphism. Subgraph isomorphism asks whether a query graph Q exists as a subgraph anywhere within the reference graph G [74] and is also NP-Complete [75]. Resultantly, most approaches to SGM use heuristic approaches. Simple approaches slowly explore the graph from a given starting vertex, adding nodes that match the query pattern until a constraint is violated and then backtracking to the last known 'good' state. Backtracking approaches end up traversing a lot of edges in their approach and can lead to high numbers of memory accesses in large graphs as edges are traversed to vertices that do

not share spatial locality in memory. The subgraph isomorphism problem can be simplified with the addition of attribute information like vertex labels that allow for more aggressive pruning of the search space.

Reachability and Searching. The *Link Analysis* and *Kill Chain* use cases both can be characterized as a problem of *Graph Search*, and the special case of *Reachability*. These search-based queries form the majority of queries performed by the I&S community, asking the questions of *'What is happening with this entity'* (neighborhood search) and *'Is there a connection between these two entities'* [2]. They are also tractable problems, with graph searches being primitive operations that are among the first taught to computing students around the world.

The neighborhood search is implemented with a *Breadth First Search* (BFS) algorithm. Most BFS implementations pick a start vertex and *expand* it by adding all neighbors (vertices connected by edges) to a queue. It then takes a vertex from the front of the queue, checks if it is the target of the search and if not *expands* that vertex to the end of the queue. BFS can be constrained with a depth limit and terminate after a given number of traversals. The result of a depth-constrained search is the collection of vertices that form the neighborhood of interest in the *'what is happening with this entity'* query. The simple BFS search requires $O(b^d)$ time and $O(b^d)$ space to execute where b is the branching factor (approximated with $\bar{K}_{out}$) and d is the depth of the solution (how many steps to take) [76].

The reachability search can also be implemented with BFS but is commonly implemented with a *Depth First Search* (DFS) to reduce memory consumption [69]. DFS also begins with a start vertex, but instead of adding the expanded nodes to a queue, they are added to a stack. At each step, the vertex on top of the stack is checked to see if it is the target, and if not its expansion is added to the stack, continuing until the whole graph has been visited or the target is found. The

simple DFS search requires $O(b^b)$ time and $O(bm)$ space to execute where b is the branching factor and m is the maximum depth of the search [76].

Entity Resolution. The *All Source Fusion* intelligence problem is largely concerned with generating and transforming collected data into usable information that can be analyzed to create intelligence. To that end, it is largely focused on deduplication, and so can be viewed as an instance of the *entity resolution* problem. The goal of entity resolution is to reconcile data references corresponding to the same real-world entity. It can be framed as a graph problem by using the references to an entity to ground entities by their relation to other entities. The logic is that possible co-referent entities will share many of the same links to other entities, so topological similarity can be leveraged in addition to attribute similarity. In addition to this approach which uses *graph clustering* [21], other approaches have used*graph embedding* [22] distance, *neural* [23] methods, and *graph-theoretic approaches* to exploit topological and attribute information [24] of static graphs to resolve entities. Work on *dynamic graphs* uses an *incremental partitioning* algorithm to re-evaluate prior resolutions as new information is added to the graph [25].

Graph clustering problems are NP-Hard, and heuristic approaches remain expensive [77]. Popular graph embedding approaches like *DeepWalk* [78] and *Node2Vec* [79] run in $O(jVj)$ and $O(b^2)$ (where B is the *branching factor*) respectively [80], but then must be processed by a MLP or Random Forest classifier. The graph-theoretic approach runs in $O(jEj^3)$, with the impact reduced by running graph clustering as a pre-processing step to 'block' the graph into smaller subgraphs to reduce processing time. All that is to say that entity resolution is heavily impacted by the size and density of the graph, and hardware acceleration may offer the best path to improving entity resolution in a time-constrained setting.

Co-movement discovery. The geospatial analysis required to identify persons and places

of interest based on reported movement patterns can be characterized as a *co-movement problem*. The co-movement problem aims to discover entities that move together in time and space. The graph-based approach to this problem builds a separate proximity graph at each time step and then uses *graph clustering* to assign proximal entities to groups. When entities share clusters across multiple timesteps they are said to be co-movers [40]. As has been discussed above, graph clustering is typically NP-Hard and heuristic approaches are sensitive to the size and density of input graphs.

Graph Neural Networks. The *Intelligence Analysis* tasks of *missing link prediction* and *anomaly detection* can be solved using *Graph Neural Networks*. GNNs are a class of neural network that operates directly on the underlying structure of the graph. Once a GNN is *trained* for a particular task, it can be used to make *Inferences*. There are typically two key operations that happen within a GNN at inference time. First, it *Samples and Aggregates* the edges to develop an embedding representation of the node neighborhoods in the graph (the topological and attribute information). Second, it *Combines* those with the learned network weights to generate the outputs that are used to make predictions about which nodes are anomalous or where a link is likely to be missing [81] Early work in GCNs showed that they could operate linearly to the number of edges in the graph [82], but in general graphs above trivial size require the use of sampling and blocking techniques to reduce the computational complexity to an acceptable level [81].

1.2.5.2 Dataset Characteristics

The majority of datasets of interest to the I&S community share four features. They are *large & scale-free*, *attributed & heterogeneous* and *dynamic* [2].

Large & Scale-Free. Large graphs can be difficult to store in memory on a single machine. While efficient representations can represent graphs with 3.5 billion vertices and 128 billion edges and execute theoretically work-efficient graph algorithms that can be used in search and community detection in a commodity system with a Terabyte of Random Access Memory (RAM) [83]. However, for larger graphs and more complicated algorithms, single-machine solutions may not be sufficient. Distributed computing has proved effective for other large-scale problems, and so is a natural path to follow for large-scale graph analysis. Unfortunately, graph algorithms perform poorly in a distributed setting. There are two main reasons. First, divining the work optimally involves partitioning the graph optimally. Graph partitioning is itself an NP-complete problem, and so heuristic partitioning requires some up-front computation, and may still yield a sub-optimal result. Kent: Cite Graph Partition NP Complete Second, because of the typically *scale-free* characteristic of I§graphs, we see that for the densely connected parts of the graph, even an optimal partitioning may have cross-partition edges. Consequently, the control mechanisms used to synchronize the processing may need to 'wait' to access information on the other machine. It also introduces delay as network bandwidth and round-trip latency degrade the communication channel further. Kent: cite distributes dramas.

Attributed & Heterogeneous. The knowledge graphs described by the I&S community are likely to be attributed, directed, and weighted and may feature multigraph and hyperedge features. A knowledge graph is typically built on a constrained ontology that defines the entities and relations at an abstract level. The information contained in vertex and edge labels is critical to answering search and reachability queries (what is the 'name' of the target vertex), can reduce complexity in subgraph isomorphism [84], improve the *effectiveness* of community detection [24] and is critical to learning useful embeddings for GNNs [81]. However, beyond just increasing

the size of the graph, the label information is typically held separately in memory from the graph topology which can impact retrieval times, and the checking and comparison of complicated node attributes can increase the computational intensity of operations.

Dynamic. Dynamic (streaming) graphs are in a constant state of change. One mechanism to handle dynamic graphs is to take 'snapshots' at a point in time and process them, which can yield outdated insights from old information. Kent: Snapshots for dynamic

Other approaches use incremental algorithms to update the results of the analysis as data is added or removed, with successful application in the entity resolution [25] and ... domains. Kent: Add cites for increme

1.2.6 Processing Bottlenecks

Kent: Intent is to use the exploit and elevate framwework i introduce here to taxonomize the acceleration methods, probably generate a table or taxonomic graph of them for a pretty visual.

A *bottleneck*, or *system constraint* is any resource whose capacity is equal to or less than the demand placed on it [85]. To improve the overall performance of a system, we must identify the bottleneck, exploit that bottleneck to maximize performance with the resources at hand, subordinate the rest of the system to that bottleneck, improve the bottleneck and then re-evaluate the system to find the new bottleneck [86]. This section reviews the prior work in identifying the bottlenecks known to occur during graph processing, setting the scene for the discussion of using hardware accelerators to *exploit* and *elevate* those constraints in section 1.3.

Graph processing began, as most computer science problems do, as a math problem–the Königsberg bridge problem–asking if the seven bridges of Königsberg could be traversed in a single trip without doubling back, ending back at the start point. Euler's answer to this problem

is generally heralded as the birth of graph theory [87]. The early computational approaches to graph problems focused on sequential algorithms on single machines. The rise of the 'big-data' era has seen graphs increase in size from the four vertexes and seven edges of the Königsberg bridge problem to billions or trillions of nodes and edges.

The increased graph size has flow-on effects. Large graphs quickly became unable to fit in the memory of a single machine, and even where they could because of a powerful machine or a compressed graph representation, sequential algorithms were too slow to process the graph in a reasonable time. Distributed computing offered a promising path to *elevate* the single machine capacity bottleneck, but problems of how to efficiently partition graphs and synchronize operations on those partitions for distributed processing imposed new constraints, with network bandwidth, network latency and stalled cycles waiting for synchronization degrading performance.

As computing capacity increased, *elevating* the memory bottleneck, shared memory parallelism for graph processing gained traction. With viable shared-memory parallel solutions emerging, the focus of the research switched to improving the efficiency of programming frameworks and the processing itself to *exploit* these gains. The large number of graph-processing frameworks that exist 'to abstract the complexity of implementing complicated parallel graph applications' evidences the growing demand for graph processing. Realizing that the performance of most graph processing algorithms is dependent on the shape of the input data drove a wave of efforts to *exploit* the interaction of the algorithm with the representation by re-ordering the graph as a pre-processing step, scheduling the order that vertices are ordered, and partitioning the graph so subgraphs can fit into cache.

Deeper analysis shows that graph algorithms are almost universally memory-bound, except for some distributed implementations which are network-bound. The constraints are commonly

attributed to memory bandwidth and memory latency emerging from poor exploitation of the cache hierarchy and pre-fetching optimizations by graphs on CPU-based single machines, and a high communication-computation ratio for parallel (distributed and shared-memory) implementations. Hardware acceleration is a natural step to *elevate* both local and distributed approaches. Representing graphs as matrices can allow us to *exploit* the significant memory bandwidth available on GPUs, though sparsity quickly becomes a problem for the efficiency of both representation and computation, as well as incurring overhead for copying the graph data from main memory to the much more limited on-chip GPU memory. Other approaches use FPGAs and ASICs to *elevate* the computation bottlenecks in graph processing. However, to maximize efficiency, they need to be hand-tuned to both the distribution of the input graph and the algorithm being deployed on the system.

There is a particular emphasis on accelerating GNN hardware as a special case of graph processing made more difficult by the *hybrid* execution patterns that include both the sparse, irregular memory accesses of graph processing and the dense, regular computation of neural networks. Taken together we can see that the efficient *general purpose* processing of graph data structures is an open problem despite decades of work to *exploit* graph processing with algorithmic and data structure innovations and *elevate* performance through parallelization and hardware acceleration. Estimates of commercially interesting graph sizes range up to 20 Terabytes (TB) [88] and as datasets become larger, approaching the exa-scale, we are likely to see an increase in the demand for accelerated, distributed graph processing [89].

The following sections summarize each of these advances in turn and highlight common themes in the limitations that are used to inform the benchmark framework presented in chapter **??**.

1.2.6.1 Memory Bottlenecks

Graph Size and Shape. The sizes of graphs have been increasing in the 'big data' era, with estimates of uncompressed commercial graph sizes exceeding 20TB [88]. Beyond the Input/Output (I/O) problems that 'big data' graphs pose in the face of no universally accepted binary exchange format [89], large graphs quickly became unable to fit in the memory of a single machine, and some algorithms scale poorly as graphs reach larger sizes [90]. When graphs are unable to fit in memory, the choices become *distributing* the computation, or *compressing* the representation [59, 91]. Popular compressed representations like the CSR/CSC formats will become hyperspace when graph decomposition techniques are employed [89], so more aggressive compression is required to make it a useful tactic. Unfortunately, compression is not viable for sequential algorithms, as the encoding/decoding cost for the compressed representation tends to outweigh the benefit of the smaller representation [59], forcing the use of parallel implementations to 'hide' the cost. Shared memory parallelism has proven able to represent and process graphs with over 3.5 billion vertices and 128 billion edges within 1 TB of RAM [66], demonstrating its viability as an alternative to distributed parallelism. Together, the size and shape of the input graph, combined with the algorithm being executed has a significant impact on computing efficency [92, 93], and so different algorithmic approaches to the same problem may need to be used depending on if the graph is *random, scale-free* or *mesh* [94].

Memory Boundedness. Graph algorithms are widely understood to be *memory bound* [95]. They are *Data Intensive* algorithms, rather than the *Compute Intensive* algorithms that modern general-purpose and GPU-accelerated systems are optimized for [96]. A program is *memory bound* when the processor is not fully utilized, typically because it is stalled waiting for data

to be retrieved from memory. The *memory bandwidth* of a system is commonly identified as the source of memory-boundedness for both *traversal* and *computationally intensive* graph algorithms [60]. Memory bandwidth is a pressure point because the large number of irregular memory accesses prevent the effective use of the cache hierarchy and because very little computation is performed on each vertex when loaded from memory [97].

The Memory Hierarchy. General-purpose architectures alleviate memory bandwidth constraints with the caching system in the memory hierarchy [1]. Caches are under-utilized in graph processing systems because of irregular memory accesses, resulting in a high cache miss rate [98] that has been measured as causing $60 - 80\%$ of cycles to stall waiting for memory accesses [99]. The ineffectiveness of pre-fetching data in graph processing puts additional pressure on the cache hierarchy. Only small portions of each cache line contain the data needed for the next computation step, wasting much of the memory bandwidth that is used to load data into the cache [63,88,100]. The 'wasted space' in the cache lines is because of a lack of *spatial locality* in graph data structures, which is generally true for many graph types, but particularly *mesh* graphs with large diameters and a low mean degree [63, 101].

Spatial Locality. *Spatial locality* is the idea that when data is needed for processing, data stored near it in memory is also likely to be needed for processing soon, and so proximal memory blocks can be pre-emptively fetched to the cache for quick access [1]. In figure 1.13, the vertices are stored in memory in *vertex-order*, that is, written to memory in the order they are created (irrespective of edges). Assuming we can load a block of 3 contiguous vertices at a time from memory to a simple single-level cache as an example, the impact that poor *spatial locality* is evident. In most BFS implementations, the vertices linked by outgoing edges from the source vertex (**A** for this example) are added to a queue, with each being processed in turn until the

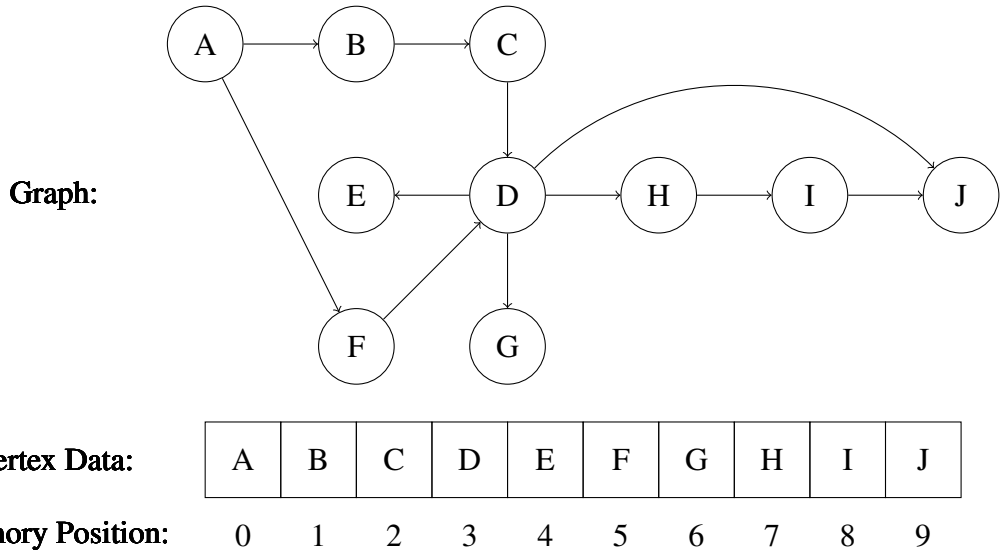


Figure 1.13: A simplified representation of a direct graph in vertex-order where each vertex is added to memory in the order it was created (here, alphabetically). Edges are not represented here.

target vertex is found. So, for the graph in figure 1.13, we would load the first three vertices from memory (**A, B** and **C**) to the cache for quick retrieval. The first 'expansion' of **A** enqueues the pointers to **B** and **F**. **B** is already in the cache and the 'cache hit' when it is dequeued for expansion means that the vertex is quickly retrieved, exploiting the spatial locality of **A** and **B** in this instance. The processer expands **B** to add the pointer to its neighbor **C** to the queue. When it comes time to expand **F**, however, there is a 'cache miss'. **F** is not in the cache, so the processor must now wait for it to be loaded from main memory, incurring a penalty proportional to the latency of retrieving the vertex from main memory plus the time it takes to load the whole block given memory bandwidth constraints. For simplicity, let us now assume that we have loaded another 3 vertices starting from **F** (**F, G, H**) to replace the content of the cache and expand **F**, adding its neighbor **D** to the queue. **C** is now at the front of the queue but has been bumped out of the cache by the load of (**F, G, H**) so the processor must wait again for **C** to be loaded from main

memory before it can be expanded. The problem continues throughout the algorithm, effectively 'chasing pointers' all over the main memory, the 'irregular memory accesses' that cause so much difficulty in graph processing [95]. The 'pointer chasing' behavior here happens because of the poor *spatial locality* in this graph. Put simply, the vertices that are close to each other in the graph topologically are not close to each other in memory. It also shows how only a small percentage of each cache line is used in each step, putting pressure on memory bandwidth to load the entire block with very little use of what is loaded. The irregular memory access problem is exacerbated in larger graphs, where it is more likely that the vertex list does not fit in the cache and will have to access the main memory to retrieve the data [102], reflected empirically in a decreasing *instructions per miss* rate on larger graphs [95].

Temporal Locality. There is, however, some natural locality in *scale-free* graphs that can be readily exploited. Consider figure 1.13 again. While it isn't a scale-free graph in any real sense, vertex **D** has a much higher *degree* than the other vertices. To *exploit* the *temporal locality* the system should cache nodes with higher degrees, as they are the ones most likely to be required by a graph algorithm. So in a scale-free graph where most of the edges are owned by few vertices, there is latent temporal locality [95].

Memory Bandwidth Utilization. Naïve 'flat' parallel processing implementations exacerbate memory bandwidth issues and impose a scaling limit on the number of processors as they for limited memory bandwidth [92]. The observation is more concretely characterized by a decay factor of $P^{\frac{1}{2}}$ where P is the number of processors [103] because of the unfavorable communication-computation ratio. However, studies of memory bandwidth utilization in general show that around 95% of production systems utilize less than one-third of their memory bandwidth [104]. Graph-specific studies indicate that the existing bandwidth is poorly utilized

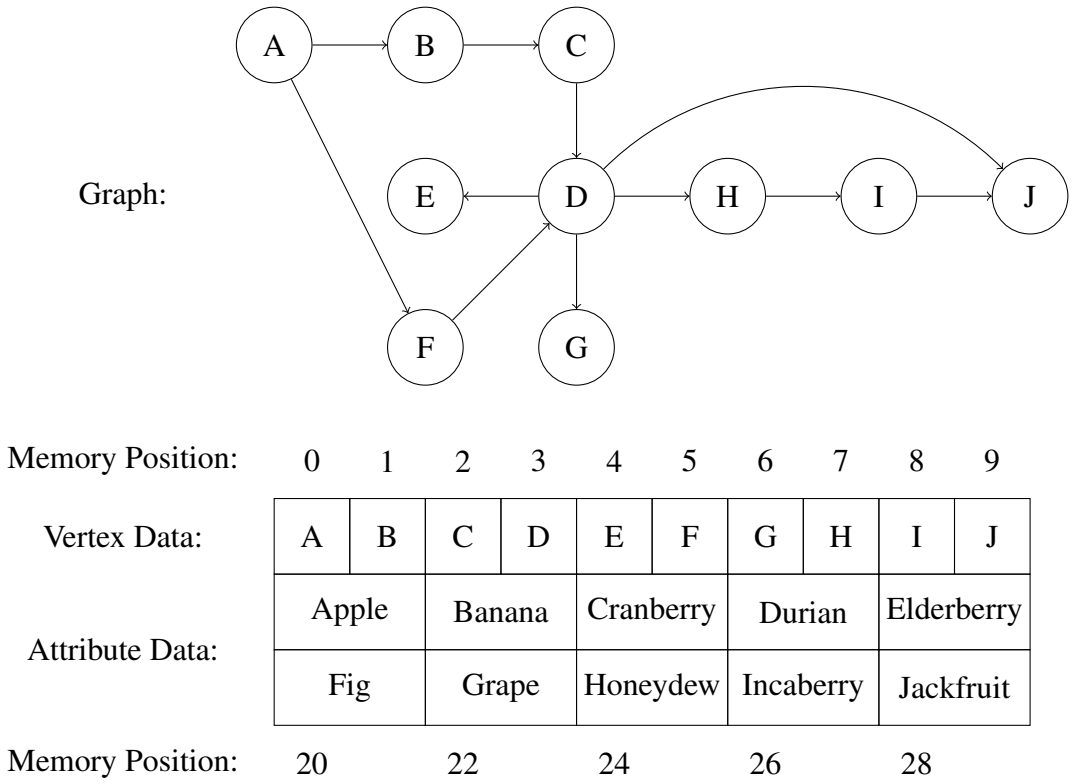


Figure 1.14: A simplified representation of a direct graph in vertex order where each vertex is added to memory in the order it was created with attributes stored separately. Pointers from topology to attributes not shown for clarity, but correspond to first letter of attribute.

when the instruction window size cannot hold enough instructions to account for cache-missing instructions [95]. While experiments on GPUs have shown that *elevating* the memory bandwidth can increase throughput [56] through brute-force alone, it is insufficient to just increase the bandwidth, and more sophisticated methods are needed.

Memory Latency & Attributed Graphs. Contributing to the memory bandwidth issue, memory latency has a significant negative impact on system throughput [95]. The lack of locality combined with the data intensity of graph operations results in the latency of memory accesses overwhelming even highly optimized pipelined architectures [96]. The latency problem is exacerbated by attributed graphs. It is common for the topological and attribute data of a graph to be stored separately in memory [105]. Figure 1.14 expands on the example from figure 1.13 by adding in vertex attributes. The attributes are stored at separate memory locations, with pointers from the topological vertex (memory positions $1-9$) to their respective attributes (memory positions $10-29$). Attributes consume more memory so more memory bandwidth is required to load them. The heterogeneous data types being loaded in attributed graphs (like knowledge graphs) have been shown to inflict significant Dynamic Random Access Memory (DRAM) accesses and hence, increase latency and memory bandwidth pressure [106]. To illustrate why, again consider a BFS that is searching for the '**Grape**' vertex. Each time a vertex is dequeued, its attribute must be retrieved from memory to see if it is the target vertex. In the worst case, this is a memory access each time a vertex is checked, or, if reducing the latency of the comparison by pre-fetching the attribute along with the vertex, the cache block of 3 quickly becomes filled with the vertex plus its attribute, exacerbating the spatial locality problem.

Overall, the processing of a graph is highly dependent on the shape of its input data. *Scale-free* graphs have better temporal locality than *mesh* graphs, but both struggle with optimizing for

spatial locality. Combined, the lack of spatial and temporal locality results in poor utilization of the memory hierarchy in the average case [107]. The ineffective use of the memory hierarchy is exacerbated by the increasing sizes of graphs, with the largest not fitting in the memory of a single machine, and a known negative correlation between graph size and cache utilization. Graph processing is memory-bound and finding ways to *Exploit* and *Elevate* remain important research priorities.

1.2.6.2 Communication Bottlenecks

Parallel approaches to graph processing are essential to addressing the memory and computation bottlenecks, but they incur particularly nasty communication bottlenecks because of the poor *Communication to Computation* ratio causing them to become network-bound when algorithms need to process the entire vertex list [60] or when the graph is sub-optimally partitioned between workers [108]. It is seen as the primary bottleneck impeding the scaling of graphs processing to exa-scale [89]. Communication bottlenecks are present in all parallel graph algorithm implementations, stemming from the need the irregular communication-computation patterns that manifest as a large number of main memory accesses in the shared-memory paradigm and the large volume of inter-processor messages in the distributed paradigm [95]. In general, message-passing is more expensive than memory accesses [95] and given that very large graphs can be processed in shared-memory parallel environments [66, 95], the conventional wisdom is to avoid distributed processing. Further, parallel algorithms need to enforce coarse and fine-grain synchronization [88] which can impose additional delay as processors are forced to wait during unbalanced computation when there is high contention for shared vertices [108].

Architectures. Graph processing workloads have been shown to run poorly on general-purpose parallel processors because the computing patterns of graph processing do not re ect the conventional highly localized workloads they are optimized to support [57, 98]. Known issues involve incurring overhead when the parallel framework enforces explicit function calls to exchange data between worker nodes [96] and that general-purpose instruction sets like x86 do not the inter-processor traf c with instructions [63]. One of the common thrusts in the discussion of parallel graph processing architectures is that a domain-speci c architecture is needed to achieve optimal performance.

1.2.6.3 Computation Bottlenecks

While graph processing is typically memory-bound, there are computational bottlenecks that can occur Load-balancing is a key challenge, particularly when parallelizing graph processing and working with graphs with heavilily skewed degree distributions [88]. The irregular structure of graphs makes it hard to predict what work needs to be completed before execution time. For example, many graph algorithms iterate until convergence, however, the number of iterations required to converge individual vertices or subgraphs can vary signi cantly, blocking other workers while they wait for the over-loaded worker to nish [109].

Branching. Graph processing is challenging for traditional architectures [110]. In addition to the pre-fetching and cache issues already discussed, graphs can make branch prediction and out-of-order processing optimizations ineffective. Graph applications heavily use branches because future processing depends heavily on the structure and order of the input data that cannot be known until input time. The branching operations are data dependent, irregular and so are